

EE-2003

Computer Organization and Assembly Language

Week 14 – Lecture 28

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Chapter 7: String Instructions
- 3 Why Use REP? Loops vs String Instructions
- 4 STOS Instruction
- 5 Example: Clearing the Screen

Quick Review: What was in the Last Lecture?

- Screen Location Calculation
- Quick review of Chapter 6

Introduction to String Instructions

- We now explore more powerful 8088 instructions that process **blocks of data**.
- These are called **string instructions** or **block instructions**.
- They work on sequences of bytes/words using SI (source) and DI (destination).
- When used with the **REP prefix**, they outperform normal loops.
- Key string instructions:
 - MOVS – Move block
 - STOS – Store value
 - LODS – Load value
 - SCAS – Scan string
 - CMPS – Compare strings

Common Properties of String Instructions

- Work with **SI** (DS:SI) and **DI** (ES:DI).
- Operate in byte (B) or word (W) mode.
- Automatically increment or decrement:
 - **CLD** – Auto increment (forward)
 - **STD** – Auto decrement (backward)
- Can process entire blocks using:
 - **REP** – repeat CX times
 - **REPE/REPZ** – repeat while equal
 - **REPNE/REPNZ** – repeat while not equal

Traditional Loops vs String Instructions

Software loop (slow):

- Each iteration executes several instructions:
- Load → Store → INC → CMP → JNZ
- High instruction-fetch overhead.

String instructions (fast):

- Only one instruction is fetched (REP STOSW, REP MOVSW, etc).
- Hardware internally repeats operation **CX times**.
- No explicit increment, comparison, or branching.

Efficiency Example (approximate cycle counts)

- Loop version: 6000 cycles for clearing screen.
- REP STOSW: 2000 cycles.

STOS (Store String)

- Stores AL or AX into the location ES:DI.
- Updates DI automatically (inc/dec based on DF).
- Perfect for:
 - Clearing screen
 - Filling memory with constant pattern
- When combined with REP, it clears or fills entire blocks fast.

Example: Clear Screen with REP STOSW

```
; clear screen using string instructions
[org 0x0100]
jmp start ; declare the main function by yourself

clrscr:
    push es
    push ax
    push cx
    push di
mov ax, 0xb800
mov es, ax          ; ES = video memory segment
xor di, di          ; start at offset 0
mov ax, 0x0720       ; space char in normal attribute
mov cx, 2000         ; number of screen words
cld                 ; auto-increment mode
rep stosw           ; clear screen

    pop di
    pop cx
    pop ax
    pop es
ret
```

Understanding the Example

- $0xB800:0000$ = start of color text video memory.
- Each character cell = 2 bytes (character + attribute).
- $0x0720$ = space ($0x20$) + normal attribute ($0x07$).
- **2000 words** = full screen ($80 \times 25 \times 2$ bytes).
- **REP STOSW** performs:

$AX \rightarrow ES : DI$

automatically repeated 2000 times.

Why This Is So Fast

- Loop version performs:
 - Write $\rightarrow$ INC DI $\rightarrow$ DEC CX $\rightarrow$ JNZ
- String version performs:

REP STOSW $\Rightarrow$ hardware repeats the loop internally

- Fewer instruction fetches $\rightarrow$ faster execution.
- Cleaner, smaller, and more efficient code.

Key Takeaways

- String instructions simplify operations on large blocks of memory.
- SI and DI act as automatic memory pointers.
- DF selects direction: forward/backward scanning.
- REP executes loops in hardware — not software.
- REP STOSW provides a fast method to clear the screen.

Thank You!

Any Questions?

- Today we saw how powerful the 8088 string instructions are for handling blocks of data.
- The STOS and MOVS instructions automate memory access using SI and DI.
- The DF flag controls direction: CLD for forward and STD for backward processing.
- Using the REP prefix allowed the CPU to repeat operations internally, making them much faster than software loops.
- The clear-screen example showed how REP STOSW can update all 2000 screen words efficiently.

“String instructions show how hardware-level repetition can replace long software loops with a single powerful command.”